Data Structure Lab Program List

1. *Implementation of Array operations i) insertion ii) deletion iii) merging iv) traversal*

```
#include<stdio.h>
int n, a[100];
int main()
{
        void merge(int a[], int);
        void traverson(int a[], int);
        int insertion(int a[],int);
        void deletion(int a[],int);

        int i,fun,op;

        printf("Enter total elements of the array : ");
        scanf("%d",&n);
        printf("Enter %d elements : \n",n);
        for (i=0;i<n;i++)
         {
                scanf("%d",&a[i]);
        }

  do
  {

  printf("\nEnter the operation you want to execute : \n 1 - Insertion\n 2 - Deletion\n 3 - Traverson\n 4 - Merging Two arrays\n 5 - Exit \n\n ");
        scanf("%d",&op);
```

```
        switch(op)
        {
             case 1: insertion(a,n);
                  break;
             case 2: deletion(a,n);
                  break;
             case 3: traverson(a,n);
                  break;
             case 4: merge(a,n);
                  break;
             case 5: printf("Exit Program\n");
                  break;
             default : printf("Invalid Selection\n");
                  break;
        }

   } while (op != 5);

         return 0;
}

void merge(int a[],int n)
{
   int j,n2,i;

   printf("\nMERGING TWO ARRAYS\n");


   printf("Enter total elements of the second array : ");
   scanf("%d",&n2);
   printf("Enter the elements of second array : \n");
   for (j=n;j<n+n2;j++)
   {
        scanf("%d",&a[j]);
   }
```

```
  printf("The updated array is : \n");
  for (i=0;i<n+n2;i++)
  {
      printf("%d\n",a[i]);
  }
}
void traverson(int a[],int n)
{
  int i;
  printf("\nTRAVERSON (Display Entered Array)\n");
  printf("Your array is : \n");
        for (i=0;i<n;i++)
        {
                printf("%d\n",a[i]);
        }
}


int insertion(int a[],int n)
{
        int p,i,num,number;
  printf("\nINSERTION\n");
        printf("Please Enter the number to insert : ");
        scanf("%d",&number);
        printf("Enter the position to insert : ");
        scanf("%d",&num);
        p=num-1;
        for(i=n;i>p;i--)
        {
                a[i]=a[i-1];

        }
        a[i]=number;
        for (i=0;i<=n;i++)
        {
```

```
                printf("%d\n",a[i]);
        }
    return 0;
}

void deletion(int a[], int n)
{

    int i,p,num;
    printf("\nDELETION\n");
        printf("Element position to delete : ");
        scanf("%d",&num);
        p=num-1;
        for(i=p;i<n;i++)
        {
                a[i]=a[i+1];
        }
        for(i=0;i<n-1;i++)
        {
                printf("%d\n",a[i]);
        }

}
```

2. *Linear Search in Array*

```
#include<stdio.h>

int element_compare(int a[],int n, int num)
{
    int i,j;
    for (i=0;i<n;i++)
    {
        if(a[i]==num)
        {
```

```
                return num;
            }
        }
            return -1;
}

int main()

{
        int a[100],n,i,num,flag;
        printf("Enter total elements : ");
            scanf("%d",&n);
            printf("Enter %d elements : \n",n);
            for (i=0;i<n;i++)
            {
                    scanf("%d",&a[i]);
            }
        printf("Enter the number to search : ");
        scanf("%d",&num);
        flag=element_compare(a,n,num);
        if (flag==-1)
        {
            printf("Searching Failed \n");
        }
        else
        {
            printf("Found\n");
        }
        return 0;
}
```

3. *Binary Search in Array*

```
#include<stdio.h>
void main()
```

```
{
     int a[100],i,loc,mid,beg,end,n,flag=0,item;
     printf("Enter total elements of the array : ");
        scanf("%d",&n);
        printf("Enter %d elements : \n",n);
        for (i=0;i<n;i++)
        {
                scanf("%d",&a[i]);
        }
     printf("Enter the element to search : ");
     scanf("%d",&item);
     loc=0;
     beg=0;
     end=n-1;

     do
     {
          mid=((beg+end)/2);

          if(item==a[mid])
          {
               printf("Search is successful\n");
               loc=mid;
               printf("Position of the item : %d\n",loc+1);
               flag=flag+1;
          }
          else if (item<a[mid])
          {
               end=mid-1;
          }
          else
          {
               beg=mid+1;
          }
     } while ((beg<=end)&&(item != a[mid]));
```

```
    if (flag==0)
    {
        printf("Search is not successful\n");
    }
}
```

4.Selection Sort

```
#include<stdio.h>
#include<math.h>
void selection(int arr[], int n)
{
    int i=0,j=0,small;
    for (i=0;i<n-1;i++)
    {
        small=i; // Minimum element in unsorted array

        for (j=i+1;j<n;j++)
        {
            if (arr[j]<arr[small])
            {
                small=j;
            }
        }

        int temp=arr[small];
        arr[small]=arr[i];
        arr[i]=temp;
    }
}

void  printArr(int a[], int n)
{
```

```
     int i=0;
     for (i=0;i<n;i++)
     {
          printf("%d\n",a[i]);
     }
}

int main()
{
     int n,i=0,num;

     printf("Enter total elements of the array : ");
        scanf("%d",&num);

     int a[num];

        printf("Enter %d elements : \n",num);
        for (i=0;i<num;i++)
        {
                scanf("%d",&a[i]);
        }

     n=sizeof(a)/sizeof(a[0]);

     printf("Array Before Sorting : \n");
     printArr(a,n);

     selection(a,n);

     printf("\nAfter Sorting Array Elements\n");
     printArr(a,n);

     return 0;
}
```

5. Bubble Sort

```
#include<stdio.h>

void bubbleSort(int a[], int n)
{
     int i,j;
     for(i=0;i<n-1;i++)
     {
          int swapped=0;
          for(j=0;j<n-i-1;j++)
          {
               if (a[j]>a[j+1])
               {
                    int temp = a[j];
                    a[j]=a[j+1];
                    a[j+1]=temp;
                    swapped=1;
               }
          }
          if (!swapped)
          {
               break;
          }
     }

}

int main()
{
     int a[100],n,j;

     printf("Enter total elements of the array : ");
        scanf("%d",&n);
```

```
        printf("Enter %d elements : \n",n);
        for (int i=0;i<n;i++)
        {
                scanf("%d",&a[i]);
        }

    printf("Array Before Sorting : \n");
    for (j=0;j<n;j++)
    {
         printf("%d\n",a[j]);
    }

    bubbleSort(a,n);

    printf("\nAfter Sorting Array Elements\n");

    for (j=0;j<n;j++)
        {
                printf("%d\n",a[j]);
        }

    return 0;
}
```

6. *Insertion Sort*

```
#include<stdio.h>
void insertion_sort(int arr[],int n)
{
    int i,j,key;

    for(i=0;i<n;i++)
    {
         key=arr[i];
         j=i-1;
```

```
        while(j>=0 && arr[j]>key)
        {
            arr[j+1]=arr[j];
            j=j-1;
        }
        arr[j+1]=key;

    }

}

void printArr(int arr[],int n)
{
    for(int i=0;i<n;i++)
    {
        printf("%d\n",arr[i]);
    }
}

int main()
{
    int arr[100],i,n;

    printf("Enter the total values of the array : ");
    scanf("%d",&n);

    printf("Enter %d elements of the array : \n",n);
    for (i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }

    //n=sizeof(arr)/sizeof(arr[0]);

    printf("Sorted Array : \n");
```

```
    insertion_sort(arr,n);
    printArr(arr,n);

    return 0;
}
```

7.Matrix Addition

```
#include<stdio.h>
int main()
{
        int a[100][100],b[100][100],c[100][100],i,j,rows,columns;
        printf("Enter the number of Rows : ");
        scanf("%d",&rows);
        printf("Enter the number of Columns : ");
        scanf("%d",&columns);
    printf("Enter Matrix A elements : \n");
        for (i=0;i<rows;i++)
        {
         printf("\n");
                for (j=0;j<columns;j++)
                {
                        scanf("%d",&a[i][j]);
                }
        }

    printf("Enter Matrix B elements : \n");
        for (i=0;i<rows;i++)
        {
         printf("\n");
                for (j=0;j<columns;j++)
                {
                        scanf("%d",&b[i][j]);
                }
```

```
        }

    for (i=0;i<rows;i++)
        {
                for (j=0;j<columns;j++)
                {
              c[i][j]=a[i][j]+b[i][j];
                }
        }

    printf("Matrix A + Matrix B : \n");
    for (i=0;i<rows;i++)
        {
         printf("\n");
                for (j=0;j<columns;j++)
                {
                        printf("%d\t",c[i][j]);
                }
        }
    printf("\n");
    return 0;
}
```

8. Matrix Subtraction

```
#include<stdio.h>
int main()
{
        int a[100][100],b[100][100],c[100][100],i,j,rows,columns;
        printf("Enter the number of Rows : ");
        scanf("%d",&rows);
        printf("Enter the number of Columns : ");
        scanf("%d",&columns);
        printf("Enter Matrix A elements : \n");
```

```
for (i=0;i<rows;i++)
{
        printf("\n");
        for (j=0;j<columns;j++)
        {
                scanf("%d",&a[i][j]);
        }
}

printf("Enter Matrix B elements : \n");
for (i=0;i<rows;i++)
{
        printf("\n");
        for (j=0;j<columns;j++)
        {
                scanf("%d",&b[i][j]);
        }
}

for (i=0;i<rows;i++)
{
        for (j=0;j<columns;j++)
        {
                c[i][j]=a[i][j]-b[i][j];
        }
}

printf("Matrix A - Matrix B : \n");
for (i=0;i<rows;i++)
{
        printf("\n");
 for (j=0;j<columns;j++)
        {
                printf("%d\t",c[i][j]);
        }
```

```
        }
        printf("\n");
        return 0;
}
```

9.Matrix Multiplication

```
#include<stdio.h>
int main()
{
     int row1, col1,row2,col2,i,j;
        printf("Enter the number of Matrix A Rows : ");
        scanf("%d",&row1);
        printf("Enter the number of Matrix A Columns : ");
        scanf("%d",&col1);
     printf("Enter the number of Matrix B Rows : ");
     scanf("%d",&row2);
        printf("Enter the number of Matrix B Columns : ");
        scanf("%d",&col2);

     if (col1 != row2)
     {
          printf("Multiplication not possible Column if number of
elements of first matrix column and second matrix row not equal\n");
          return 0;
     }

     int A[row1][col1],B[row2][col2],C[row1][col2];

     printf("Enter Matrix A elements : \n");
        for (i=0;i<row1;i++)
        {
          printf("\n");
                 for (j=0;j<col1;j++)
```

```
            {
                    scanf("%d",&A[i][j]);
            }
    }

printf("Enter Matrix B elements : \n");
    for (i=0;i<row2;i++)
    {
     printf("\n");
            for (j=0;j<col2;j++)
            {
                    scanf("%d",&B[i][j]);
            }
    }

for (i=0;i<row1;i++)
    {
            for (j=0;j<col2;j++)
            {
          C[i][j]=0;
          for(int k=0;k<col1;k++)
          {
               C[i][j] = A[i][k]*B[k][j];
          }
            }
    }

printf("Multiplied Array : ");
printf("Matrix A * Matrix B : \n");
for (i=0;i<row1;i++)
    {
     printf("\n");
            for (j=0;j<col2;j++)
            {
                    printf("%d\t",C[i][j]);
```

```
                }

        }
    printf("\n");
    return 0;
}
```

10. Stack Implementation using Arrays

```
#include<stdio.h>
int stack[100],i,j,choice=0,n,top=-1;

void push();
void pop();
void show();


int main()
{
    printf("Enter the number of elements in the stack : ");
    scanf("%d",&n);

    printf("\n*******Stack Operations Using Array*******\n\n");
    while(choice!=4)
    {
        printf("\nChoose one from the below options : \n\t");
        printf("\n\t1 - Push\n\t2 - Pop\n\t3 - Show\n\t4 -
Exit\n\n");

        printf("Enter your choice : ");
        scanf("%d",&choice);

        switch(choice)
        {
```

```
              case 1: push();
                   break;
              case 2: pop();
                   break;
              case 3: show();
                   break;
              case 4: printf("Exiting from the program...\n");
                   return 0;
                   break;
              default : printf("Invalid Choice. Please enter a valid
choice\n");
                    break;
         };
     }

}



void push()
{
     int val;
     if (top==n-1)
     {
          printf("Overflow\n");
     }
     else
     {
          printf("Enter the value : ");
          scanf("%d",&val);
          top=top+1;
          stack[top]=val;
     }
}

void pop()
```

```
{
     if(top==-1)
     {
          printf("Underflow\n");
     }
     else
     {
          top=top-1;
     }
}

void show()
{
     for (i=top;i>=0;i--)
     {
          printf("%d\n",stack[i]);
     }
     if(top==-1)
     {
          printf("Stack is empty\n");
     }
}
```

11. *Postfix Expression Evaluation using Stack*

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

// Stack structure
typedef struct
{
    int items[MAX];
```

```
    int top;
} Stack;

void initStack(Stack *s)
{
    s->top = -1;
}

void push(Stack *s, int value)
{
    s->items[++s->top] = value;
}

int pop(Stack *s)
{
    return s->items[s->top--];
}

int evaluatePostfix(char *postfix)
{
    Stack s;
    initStack(&s);
    int i, op1, op2;

    for (i = 0; postfix[i] != '\0'; i++)
    {
        if (isdigit(postfix[i]))
        {
            push(&s, postfix[i] - '0'); // Convert char to int
        }
        else
        {
            op2 = pop(&s);
            op1 = pop(&s);
```

```
            switch (postfix[i])
            {
                case '+': push(&s, op1 + op2); break;
                case '-': push(&s, op1 - op2); break;
                case '*': push(&s, op1 * op2); break;
                case '/': push(&s, op1 / op2); break;
            }
        }
    }
    return pop(&s);
}

int main()
{
    char postfix[MAX];

    printf("Enter postfix expression: ");
    scanf("%s", postfix);

    printf("Result: %d\n", evaluatePostfix(postfix));

    return 0;
}
```

12. *Infix to Postfix Conversion using Stack*

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

// Stack structure
typedef struct
{
```

```
    char items[MAX];
    int top;
} Stack;

void initStack(Stack *s)
{
    s->top = -1;
}

void push(Stack *s, char c)
{
    s->items[++s->top] = c;
}

char pop(Stack *s)
{
    return s->items[s->top--];
}

char peek(Stack *s)
{
    return s->items[s->top];
}

int precedence(char op)
{
    switch (op)
    {
        case '+':
        case '-': return 1;
        case '*':
        case '/': return 2;
        case '^': return 3;
    }
    return 0;
```

```
}

void infixToPostfix(char *infix, char *postfix)
{
    Stack s;
    initStack(&s);
    int i, j = 0;

    for (i = 0; infix[i] != '\0'; i++)
    {
        if (isalnum(infix[i]))
        {
            postfix[j++] = infix[i];
        }
        else if (infix[i] == '(')
        {
            push(&s, infix[i]);
        }
        else if (infix[i] == ')')
        {
            while (peek(&s) != '(')
            {
                postfix[j++] = pop(&s);
            }
            pop(&s); // Remove '(' from stack
        }
        else
        {
            while (s.top != -1 && precedence(peek(&s)) >=
precedence(infix[i]))
            {
                postfix[j++] = pop(&s);
            }
            push(&s, infix[i]);
        }
```

```
    }

    while (s.top != -1)
    {
        postfix[j++] = pop(&s);
    }
    postfix[j] = '\0';
}

int main()
{
    char infix[MAX], postfix[MAX];

    printf("Enter infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

13. *Infix to Prefix Conversion using Stack*

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

// Stack structure
typedef struct
{
```

```
    char items[MAX];
    int top;
} Stack;

void initStack(Stack *s)
{
    s->top = -1;
}

void push(Stack *s, char c)
{
    s->items[++s->top] = c;
}

char pop(Stack *s)
{
    return s->items[s->top--];
}

char peek(Stack *s)
{
    return s->items[s->top];
}

int precedence(char op)
{
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
}

void reverseString(char *str)
{
    int len = strlen(str);
```

```
    for (int i = 0, j = len - 1; i < j; i++, j--)
    {
        char temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}

void infixToPrefix(char *infix, char *prefix)
{
    Stack s;
    initStack(&s);
    reverseString(infix);

    for (int i = 0; infix[i] != '\0'; i++)
    {
        if (infix[i] == '(')
        {
            infix[i] = ')';
        }
        else if (infix[i] == ')')
        {
            infix[i] = '(';
        }
    }

    int j = 0;
    for (int i = 0; infix[i]; i++)
    {
        if (isalnum(infix[i]))
        {
            prefix[j++] = infix[i];
        }
        else if (infix[i] == '(')
        {
```

```
            push(&s, infix[i]);
        }
        else if (infix[i] == ')')
        {
            while (peek(&s) != '(')
            {
                prefix[j++] = pop(&s);
            }
            pop(&s);
        }
        else
        {
            while (s.top != -1 && precedence(peek(&s)) >=
precedence(infix[i]))
            {
                prefix[j++] = pop(&s);
            }
            push(&s, infix[i]);
        }
    }

    while (s.top != -1)
    {
        prefix[j++] = pop(&s);
    }
    prefix[j] = '\0';

    reverseString(prefix);
}

int main()
{
    char infix[MAX], prefix[MAX];

    printf("Enter infix expression: ");
```

```
    scanf("%s", infix);

    infixToPrefix(infix, prefix);

    printf("Prefix expression: %s\n", prefix);

    return 0;
}
```

14. Simple Linear Queue Implementation using Arrays

```
#include <stdio.h>

const int QUEUE_SIZE = 100;

int queue[100];
int front = -1, rear = -1;

void enqueue(int value)
{
    if (rear >= QUEUE_SIZE - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        if (front == -1)
        {
            front = 0;
        }
        rear++;
        queue[rear] = value;
```

```
        printf("Enqueued %d\n", value);
    }
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
        front = rear = -1; // Reset queue when empty
    }
    else
    {
        int dequeued = queue[front];
        front++;
        printf("Dequeued %d\n", dequeued);
        if (front > rear) // Reset queue when empty
        {
            front = rear = -1;
        }
    }
}

void display()
{
    if (front == -1 || front > rear)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Queue elements (front to rear):\n");
        for (int i = front; i <= rear; i++)
        {
            printf("%d\n", queue[i]);
```

```
        }
    }
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
            {
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            }
            case 2:
            {
                dequeue();
                break;
            }
            case 3:
            {
                display();
                break;
            }
            case 4:
            {
```

```
                return 0;
            }
            default:
            {
                printf("Invalid choice. Please try again.\n");
                break;
            }
        }
    }

    return 0;
}
```

15. Circular Queue Implementation using Arrays

```
#include <stdio.h>

#define MAX_SIZE 5

int queue[MAX_SIZE];
int front = -1, rear = -1;

int isFull() {
    return (rear + 1) % MAX_SIZE == front;
}

int isEmpty() {
    return front == -1;
}

void enqueue(int data) {
    if (isFull()) {
        printf("Queue overflow\n");
        return;
```

```
    }
    if (front == -1) {
        front = 0;
    }
    rear = (rear + 1) % MAX_SIZE;
    queue[rear] = data;
    printf("Element %d inserted\n", data);
}

int dequeue() {
    if (isEmpty()) {
        printf("Queue underflow\n");
        return -1;
    }
    int data = queue[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX_SIZE;
    }
    return data;
}

void display() {
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    int i = front;
    while (i != rear) {
        printf("%d ", queue[i]);
        i = (i + 1) % MAX_SIZE;
    }
    printf("%d\n", queue[rear]);
```

```
}

int main() {
    int choice, data;

    while (1) {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\nEnter
your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter element to enqueue: ");
                scanf("%d", &data);
                enqueue(data);
                break;
            case 2:
                data = dequeue();
                if (data != -1) {
                    printf("Dequeued element: %d\n", data);
                }
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

16. *Find factorial of a Number using Recursion*

```
#include<stdio.h>
int fact(int n)
{
     if (n==0)
     {
          return 1;
     }
     else
     {
          return n*fact(n-1);
     }
}

int main()
{
     int n,f;
     printf("Enter a number to find factorial : ");
     scanf("%d",&n);
     f=fact(n);
     printf("Factorial = %d\n",f);
     return 0;
}
```

17. Generate Fibonacci Series using Recursion.

```
#include <stdio.h>

int fibanocci(int n)
{
     if (n==0)
          return 0;
     else if (n==1)
          return 1;
```

```
    else
        return fibanocci(n-1)+fibanocci(n-2);
}

int main()
{
    int n,i;
    printf("Enter the number of terms : ");
    scanf("%d",&n);

    printf("Fibonacci Series : ");
    for (i=0; i<n; i++)
    {
        printf("%d ",fibanocci(i));
    }
    printf("\n");
    return 0;
}
```

18. *Sparse Matrix 3-Tuple Representation*

```
#include <stdio.h>
#define srow 50
#define mrow 20
#define mcolumn 20

/*Begin of main*/
int main()
{
    int mat[mrow][mcolumn], sparse[srow][3];
    int i, j, nzero = 0, mr, mc, sr, s;

       //taking inputs
    printf("Enter number of rows : ");
```

```
     scanf("%d", &mr);
     printf("Enter number of columns : ");
     scanf("%d", &mc);

     for (i = 0; i < mr; i++)
     for (j = 0; j < mc; j++)
     {
            //taking inputs of rows and columns
            printf("Enter element for row %d,column %d : ", i + 1, j +
1);
            scanf("%d", &mat[i][j]);
        }

    //printing entered matrix
    printf("Entered matrix is : \n");
     for (i = 0; i < mr; i++)
     {
          for (j = 0; j < mc; j++)
          {
                    printf("%6d", mat[i][j]);
                    if (mat[i][j] != 0)
                    nzero++;
          }
          printf("\n");
     }

     sr = nzero + 1;
     sparse[0][0] = mr;
     sparse[0][1] = mc;
     sparse[0][2] = nzero;
     s = 1;

     for (i = 0; i < mr; i++)
        for (j = 0; j < mc; j++)
     {
```

```
        if (mat[i][j] != 0)
            {
                sparse[s][0] = i + 1;
                sparse[s][1] = j + 1;
                sparse[s][2] = mat[i][j];
                s++;
            }
    }

    //printing sparse matrix
    printf("Sparse matrix is :\n");
    for (i = 0; i < sr; i++)
    {
        for (j = 0; j < 3; j++)
            printf("%5d", sparse[i][j]);
            printf("\n");
    }
}
```

19. *Singly Linked List Implementation*

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = data;
    n->next = NULL;
    return n;
```

```
}

void insertAtFirst(struct Node** h, int d) {
    struct Node* n = createNode(d);
    n->next = *h;
    *h = n;
}

void insertAtEnd(struct Node** h, int d) {
    struct Node* n = createNode(d);
    if (!*h) {
        *h = n;
        return;
    }
    struct Node* t = *h;
    while (t->next) t = t->next;
    t->next = n;
}

void insertAtPosition(struct Node** h, int d, int p) {
    struct Node* n = createNode(d);
    if (p == 0) {
        insertAtFirst(h, d);
        return;
    }
    struct Node* t = *h;
    for (int i = 0; t && i < p - 1; i++) t = t->next;
    if (!t) {
        printf("Out\n");
        free(n);
        return;
    }
    n->next = t->next;
    t->next = n;
}
```

```
void deleteFromFirst(struct Node** h) {
    if (!*h) {
        printf("Empty\n");
        return;
    }
    struct Node* t = *h;
    *h = t->next;
    free(t);
}

void deleteFromEnd(struct Node** h) {
    if (!*h) {
        printf("Empty\n");
        return;
    }
    struct Node* t = *h;
    if (!t->next) {
        free(t);
        *h = NULL;
        return;
    }
    while (t->next->next) t = t->next;
    free(t->next);
    t->next = NULL;
}

void deleteAtPosition(struct Node** h, int p) {
    if (!*h) {
        printf("Empty\n");
        return;
    }
    if (p == 0) {
        deleteFromFirst(h);
        return;
```

```
    }
    struct Node* t = *h;
    for (int i = 0; t && i < p - 1; i++) t = t->next;
    if (!t || !t->next) {
        printf("Out\n");
        return;
    }
    struct Node* n = t->next->next;
    free(t->next);
    t->next = n;
}

void print(struct Node* h) {
    while (h) {
        printf("%d -> ", h->data);
        h = h->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;
    int choice, data, pos;

    while (1) {
        printf("\n1.Insert First 2.Insert End 3.Insert Pos 4.Delete
First 5.Delete End 6.Delete Pos 7.Print 8.Exit\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Data: ");
                scanf("%d", &data);
                insertAtFirst(&head, data);
                break;
```

```
            case 2:
                printf("Data: ");
                scanf("%d", &data);
                insertAtEnd(&head, data);
                break;
            case 3:
                printf("Data: ");
                scanf("%d", &data);
                printf("Pos: ");
                scanf("%d", &pos);
                insertAtPosition(&head, data, pos);
                break;
            case 4:
                deleteFromFirst(&head);
                break;
            case 5:
                deleteFromEnd(&head);
                break;
            case 6:
                printf("Pos: ");
                scanf("%d", &pos);
                deleteAtPosition(&head, pos);
                break;
            case 7:
                print(head);
                break;
            case 8:
                return 0;
            default:
                printf("Invalid\n");
        }
    }
}
```

20. Doubly Linked List Implementation

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

void insertAtBeginning(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    newNode->next = *head;
    (*head)->prev = newNode;
    *head = newNode;
}

void insertAtEnd(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
```

```
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}

void insertAtPosition(Node** head, int data, int position) {
    if (position < 1) {
        printf("Position should be >= 1.\n");
        return;
    }
    if (position == 1) {
        insertAtBeginning(head, data);
        return;
    }
    Node* newNode = createNode(data);
    Node* temp = *head;
    for (int i = 1; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position greater than the number of nodes.\n");
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}
```

```
void deleteAtBeginning(Node** head) {
    if (*head == NULL) {
        printf("The list is already empty.\n");
        return;
    }
    Node* temp = *head;
    *head = (*head)->next;
    if (*head != NULL) {
        (*head)->prev = NULL;
    }
    free(temp);
}

void deleteAtEnd(Node** head) {
    if (*head == NULL) {
        printf("The list is already empty.\n");
        return;
    }
    Node* temp = *head;
    if (temp->next == NULL) {
        *head = NULL;
        free(temp);
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->prev->next = NULL;
    free(temp);
}

void deleteAtPosition(Node** head, int position) {
    if (*head == NULL) {
        printf("The list is already empty.\n");
        return;
```

```
    }
    Node* temp = *head;
    if (position == 1) {
        deleteAtBeginning(head);
        return;
    }
    for (int i = 1; temp != NULL && i < position; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position is greater than the number of nodes.\n");
        return;
    }
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }
    free(temp);
}

void printListForward(Node* head) {
    Node* temp = head;
    printf("Forward List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void printListReverse(Node* head) {
    Node* temp = head;
    if (temp == NULL) {
```

```
        printf("The list is empty.\n");
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    printf("Reverse List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}

int main() {
    Node* head = NULL;
    int choice, data, position;

    while (1) {
        printf("\n1. Insert at Beginning\n2. Insert at End\n3. Insert
at Position\n4. Delete at Beginning\n5. Delete at End\n6. Delete at
Position\n7. Print Forward\n8. Print Reverse\n9. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: printf("Enter data: "); scanf("%d", &data);
insertAtBeginning(&head, data); break;
            case 2: printf("Enter data: "); scanf("%d", &data);
insertAtEnd(&head, data); break;
            case 3: printf("Enter data: "); scanf("%d", &data);
printf("Enter position: "); scanf("%d", &position);
insertAtPosition(&head, data, position); break;
            case 4: deleteAtBeginning(&head); break;
            case 5: deleteAtEnd(&head); break;
```

```
            case 6: printf("Enter position: "); scanf("%d",
&position); deleteAtPosition(&head, position); break;
            case 7: printListForward(head); break;
            case 8: printListReverse(head); break;
            case 9: exit(0);
            default: printf("Invalid choice.\n");
        }
    }
    return 0;
}
```

21. Circular Linked List Implementation.

```
#include<stdio.h>
#include<stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head;

void beginsert();
void lastinsert();
void randominsert();
void begin_delete();
void last_delete();
void random_delete();
void display();
void search();

void main() {
```

```
    int choice = 0;
    while(choice != 7) {
        printf("\n*********Main Menu*********\n");
        printf("\nChoose one option from the following list ...\n");
        printf("\n==============================================\n");
        printf("\n1.Insert in beginning\n2.Insert at last\n3.Delete
from Beginning\n4.Delete from last\n5.Search for an
element\n6.Show\n7.Exit\n");
        printf("\nEnter your choice?\n");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                beginsert();
                break;
            case 2:
                lastinsert();
                break;
            case 3:
                begin_delete();
                break;
            case 4:
                last_delete();
                break;
            case 5:
                search();
                break;
            case 6:
                display();
                break;
            case 7:
                exit(0);
                break;
            default:
                printf("Please enter valid choice..");
```

```
        }
    }
}

void beginsert() {
    struct node *ptr, *temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL) {
        printf("\nOVERFLOW");
    } else {
        printf("\nEnter the node data?");
        scanf("%d", &item);
        ptr->data = item;
        if(head == NULL) {
            head = ptr;
            ptr->next = head;
        } else {
            temp = head;
            while(temp->next != head)
                temp = temp->next;
            ptr->next = head;
            temp->next = ptr;
            head = ptr;
        }
        printf("\nnode inserted\n");
    }
}

void lastinsert() {
    struct node *ptr, *temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL) {
        printf("\nOVERFLOW\n");
```

```
    } else {
        printf("\nEnter Data?");
        scanf("%d", &item);
        ptr->data = item;
        if(head == NULL) {
            head = ptr;
            ptr->next = head;
        } else {
            temp = head;
            while(temp->next != head) {
                temp = temp->next;
            }
            temp->next = ptr;
            ptr->next = head;
        }
        printf("\nnode inserted\n");
    }
}

void begin_delete() {
    struct node *ptr;
    if(head == NULL) {
        printf("\nUNDERFLOW");
    } else if(head->next == head) {
        free(head);
        head = NULL;
        printf("\nnode deleted\n");
    } else {
        ptr = head;
        while(ptr->next != head)
            ptr = ptr->next;
        ptr->next = head->next;
        free(head);
        head = ptr->next;
        printf("\nnode deleted\n");
```

```
    }
}

void last_delete() {
    struct node *ptr, *preptr;
    if(head == NULL) {
        printf("\nUNDERFLOW");
    } else if(head->next == head) {
        free(head);
        head = NULL;
        printf("\nnode deleted\n");
    } else {
        ptr = head;
        while(ptr->next != head) {
            preptr = ptr;
            ptr = ptr->next;
        }
        preptr->next = head;
        free(ptr);
        printf("\nnode deleted\n");
    }
}

void search() {
    struct node *ptr;
    int item, i = 0, flag = 1;
    ptr = head;
    if(ptr == NULL) {
        printf("\nEmpty List\n");
    } else {
        printf("\nEnter item which you want to search?\n");
        scanf("%d", &item);
        do {
            if(ptr->data == item) {
                printf("item found at location %d", i+1);
```

```
                flag = 0;
                break;
            }
            i++;
            ptr = ptr->next;
        } while(ptr != head);

        if(flag != 0) {
            printf("Item not found\n");
        }
    }
}

void display() {
    struct node *ptr;
    ptr = head;
    if(head == NULL) {
        printf("\nnothing to print");
    } else {
        printf("\n printing values ... \n");
        do {
            printf("%d\n", ptr->data);
            ptr = ptr->next;
        } while(ptr != head);
    }
}
```

22. Binary Tree Implementation- Creation, Insertion, Deletion, Display

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define MAX 25
#define max(a, b) ((a) > (b) ? (a) : (b))

struct TreeNode {
    int val;
    struct TreeNode *left, *right;
};

struct TreeNode *root = NULL;

struct TreeNode *create(int val) {
    struct TreeNode *node = malloc(sizeof(struct TreeNode));
    if (!node) return NULL;
    node->val = val;
    node->left = node->right = NULL;
    return node;
}

void insert(int val) {
    if (!root) {
        root = create(val);
        return;
    }
    struct TreeNode *q[MAX], *temp;
    int front = 0, rear = 0;
    q[rear++] = root;
    while (front < rear) {
        temp = q[front++];
        if (!temp->left) {
            temp->left = create(val);
            return;
        } else q[rear++] = temp->left;
        if (!temp->right) {
            temp->right = create(val);
            return;
```

```
        } else q[rear++] = temp->right;
    }
}

void delete(int key) {
    if (!root) return;
    if (!root->left && !root->right && root->val == key) {
        free(root); root = NULL; return;
    }
    struct TreeNode *q[MAX], *temp, *key_node = NULL, *last_parent =
NULL;
    int front = 0, rear = 0;
    q[rear++] = root;
    while (front < rear) {
        temp = q[front++];
        if (temp->val == key) key_node = temp;
        if (temp->left) { last_parent = temp; q[rear++] = temp->left;
}
        if (temp->right) { last_parent = temp; q[rear++] = temp-
>right; }
    }
    if (key_node) {
        key_node->val = temp->val;
        if (last_parent->right == temp) last_parent->right = NULL;
        else last_parent->left = NULL;
        free(temp);
    }
}

int search(int key) {
    if (!root) return 0;
    struct TreeNode *q[MAX];
    int front = 0, rear = 0;
    q[rear++] = root;
    while (front < rear) {
        struct TreeNode *t = q[front++];
```

```
        if (t->val == key) return 1;
        if (t->left) q[rear++] = t->left;
        if (t->right) q[rear++] = t->right;
    }
    return 0;
}

int height(struct TreeNode *node) {
    if (!node) return 0;
    return 1 + max(height(node->left), height(node->right));
}

void inorder(struct TreeNode *node) {
    if (!node) return;
    inorder(node->left);
    printf("%d ", node->val);
    inorder(node->right);
}

void preorder(struct TreeNode *node) {
    if (!node) return;
    printf("%d ", node->val);
    preorder(node->left);
    preorder(node->right);
}

void postorder(struct TreeNode *node) {
    if (!node) return;
    postorder(node->left);
    postorder(node->right);
    printf("%d ", node->val);
}

void levelorder() {
    if (!root) return;
```

```
    struct TreeNode *q[MAX];
    int front = 0, rear = 0;
    q[rear++] = root;
    while (front < rear) {
        struct TreeNode *t = q[front++];
        printf("%d ", t->val);
        if (t->left) q[rear++] = t->left;
        if (t->right) q[rear++] = t->right;
    }
}

int main() {
    int ch, val;
    while (1) {
        printf("\n1.Insert 2.Delete 3.Search 4.Height 5.Inorder
6.Preorder 7.Postorder 8.Levelorder 9.Exit\n> ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Value: "); scanf("%d", &val); insert(val);
break;
            case 2: printf("Delete: "); scanf("%d", &val);
delete(val); break;
            case 3: printf("Search: "); scanf("%d", &val);
printf(search(val) ? "Found\n" : "Not Found\n"); break;
            case 4: printf("Height: %d\n", height(root)); break;
            case 5: inorder(root); break;
            case 6: preorder(root); break;
            case 7: postorder(root); break;
            case 8: levelorder(); break;
            case 9: return 0;
            default: printf("Invalid\n");
        }
    }
}
```

23. Binary Search Tree Implementation- Creation, Insertion, Deletion, Traversals

```
#include <stdio.h>
#include <stdlib.h>

struct node {
  int key;
  struct node *left, *right;
};

// Create a new node
struct node *newNode(int key) {
  struct node *temp = (struct node*)malloc(sizeof(struct node));
  temp->key = key;
  temp->left = temp->right = NULL;
  return temp;
}

// Insert a node in BST
struct node *insert(struct node *root, int key) {
  if (root == NULL) return newNode(key);
  if (key < root->key)
    root->left = insert(root->left, key);
  else
    root->right = insert(root->right, key);
  return root;
}

// Find the node with minimum value
struct node *minValueNode(struct node *node) {
  struct node *current = node;
  while (current && current->left != NULL)
    current = current->left;
```

```
  return current;
}

// Delete a node in BST
struct node *deleteNode(struct node *root, int key) {
  if (root == NULL) return root;

  if (key < root->key)
    root->left = deleteNode(root->left, key);
  else if (key > root->key)
    root->right = deleteNode(root->right, key);
  else {
    if (root->left == NULL) {
      struct node *temp = root->right;
      free(root);
      return temp;
    } else if (root->right == NULL) {
      struct node *temp = root->left;
      free(root);
      return temp;
    }

    struct node *temp = minValueNode(root->right);
    root->key = temp->key;
    root->right = deleteNode(root->right, temp->key);
  }
  return root;
}

// Inorder traversal
void inorder(struct node *root) {
  if (root != NULL) {
    inorder(root->left);
    printf("%d ", root->key);
    inorder(root->right);
```

```
  }
}

// Preorder traversal
void preorder(struct node *root) {
  if (root != NULL) {
    printf("%d ", root->key);
    preorder(root->left);
    preorder(root->right);
  }
}

// Postorder traversal
void postorder(struct node *root) {
  if (root != NULL) {
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->key);
  }
}

int main() {
  struct node *root = NULL;

  // Insert nodes
  root = insert(root, 50);
  root = insert(root, 30);
  root = insert(root, 70);
  root = insert(root, 20);
  root = insert(root, 40);
  root = insert(root, 60);
  root = insert(root, 80);

  // Display all traversals before deletion
  printf("BST Traversals before deletion:\n");
```

```
  printf("Inorder   : ");
  inorder(root);
  printf("\nPreorder  : ");
  preorder(root);
  printf("\nPostorder : ");
  postorder(root);

  // Delete some nodes
  root = deleteNode(root, 20);
  root = deleteNode(root, 30);
  root = deleteNode(root, 50);

  // Display all traversals after deletion
  printf("\n\nBST Traversals after deletion:\n");
  printf("Inorder   : ");
  inorder(root);
  printf("\nPreorder  : ");
  preorder(root);
  printf("\nPostorder : ");
  postorder(root);

  return 0;
}
```